TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
HCM·UTE
HCMC University of Technology and Engineering

ARIN330585 – Trí tuệ nhân tạo

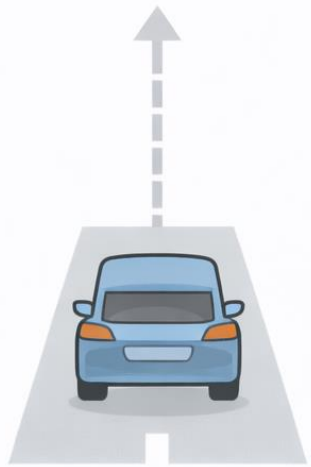
Introduction to Artificial Intelligence

NCS. Võ Long Tuấn

Chapter 7: Search in complex environments

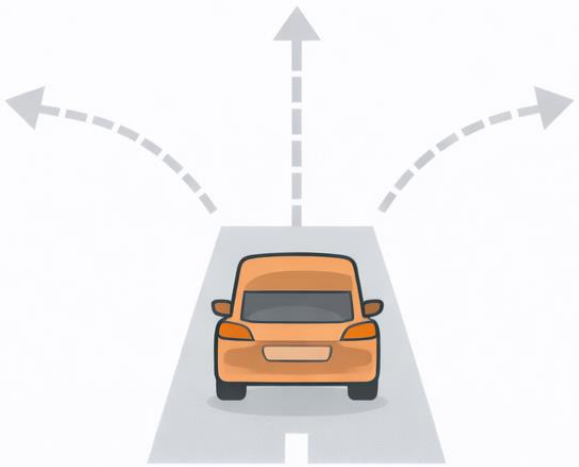
1. *Searching in stochastic environments: AND – OR search*
2. *Searching in unobservable environments*
3. *Searching in partially observable environments*
4. *Online search*

1. Searching in stochastic environments: Environments



DETERMINISTIC

Future outcome is predictable



STOCHASTIC

Future outcome is uncertain

Property 3

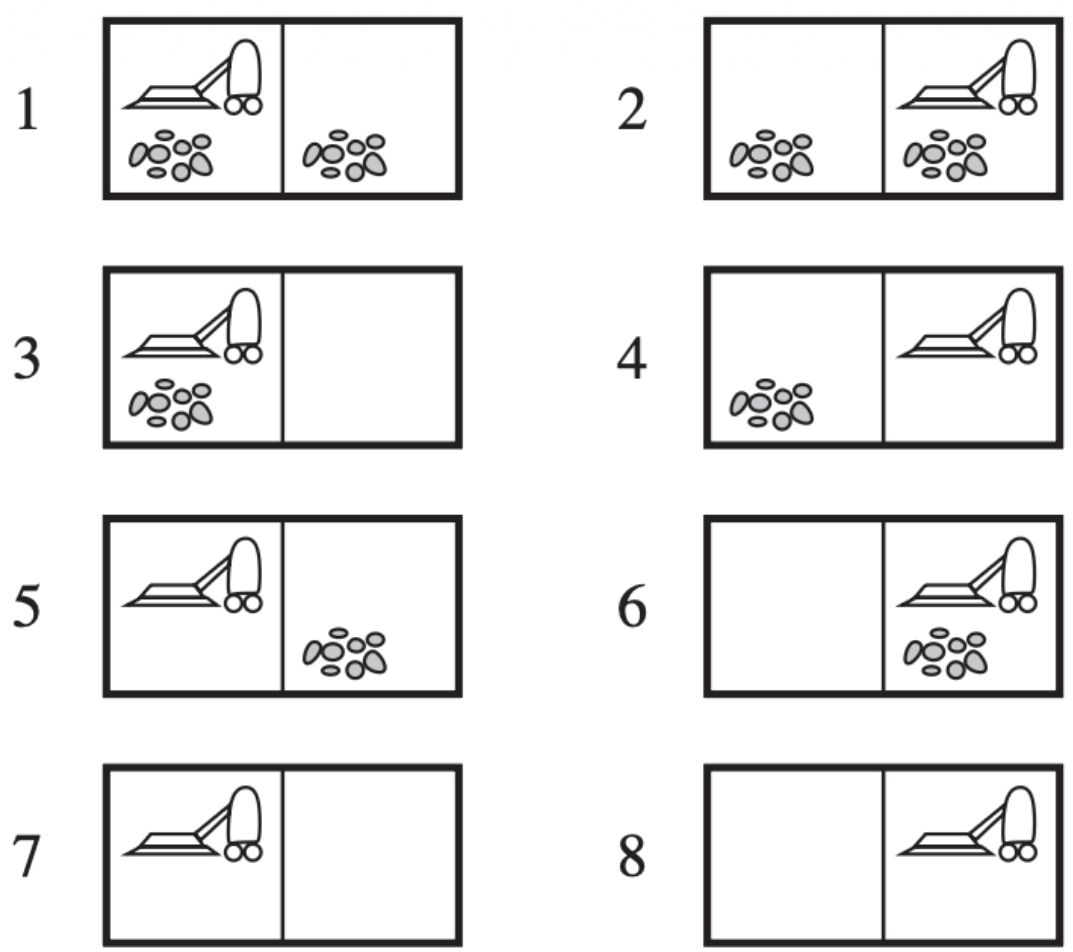
- **Deterministic:** The next state of the environment is completely determined by the current state and the agent's action.
- **Stochastic:** The next state of the environment involves randomness, so the same action can lead to different outcomes.

1. Searching in stochastic environments: Erratic vacuum machine

- Environment: (location, status)
- The state space: 8 states.
- Actions: Left, Right, Suck
- Goal: State 7 and 8.

Consider a ***stochastic environment***:

- When applied to a dirty square the action cleans the square and sometimes cleans up dirt in an adjacent square, too.
- When applied to a clean square the action sometimes deposits dirt on the carpet.



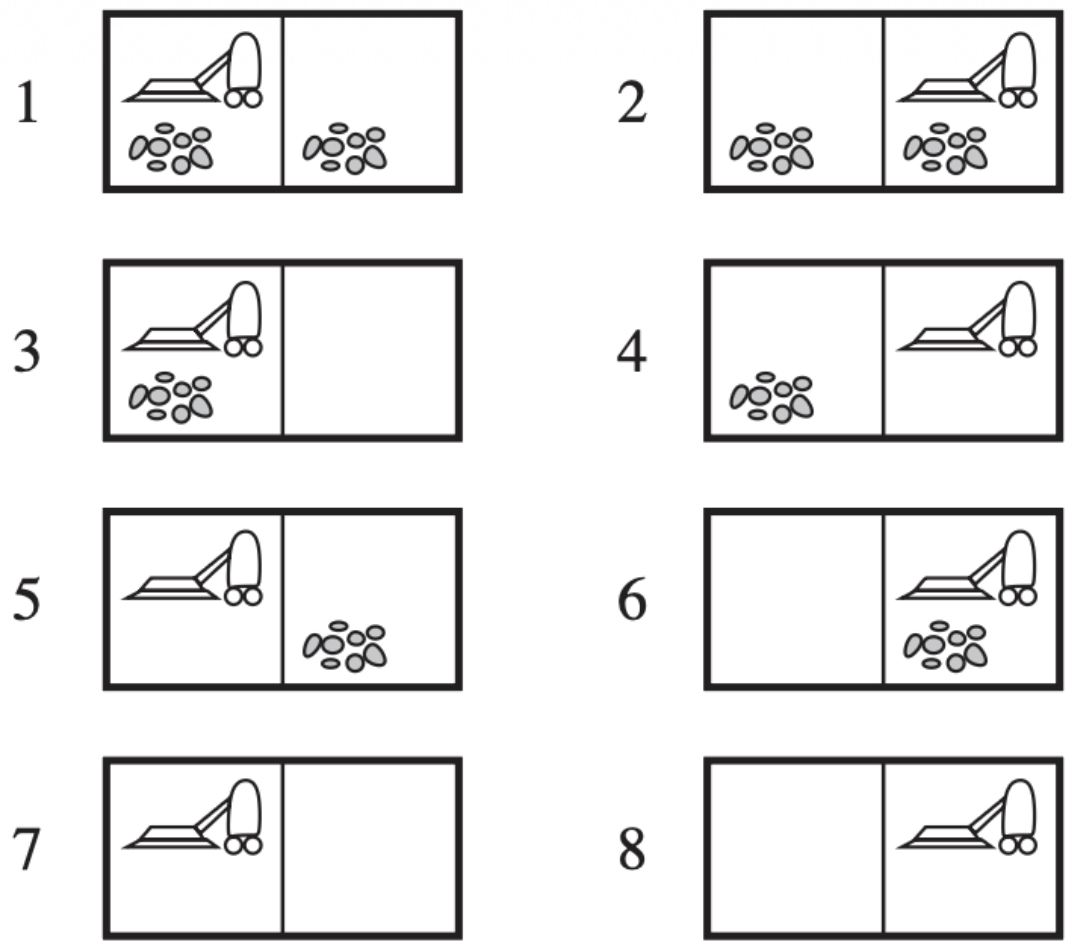
Erratic vacuum machine

- Environment: (location, status)
- The state space: 8 states.
- Actions: Left, Right, Suck
- Goal: State 7 and 8.

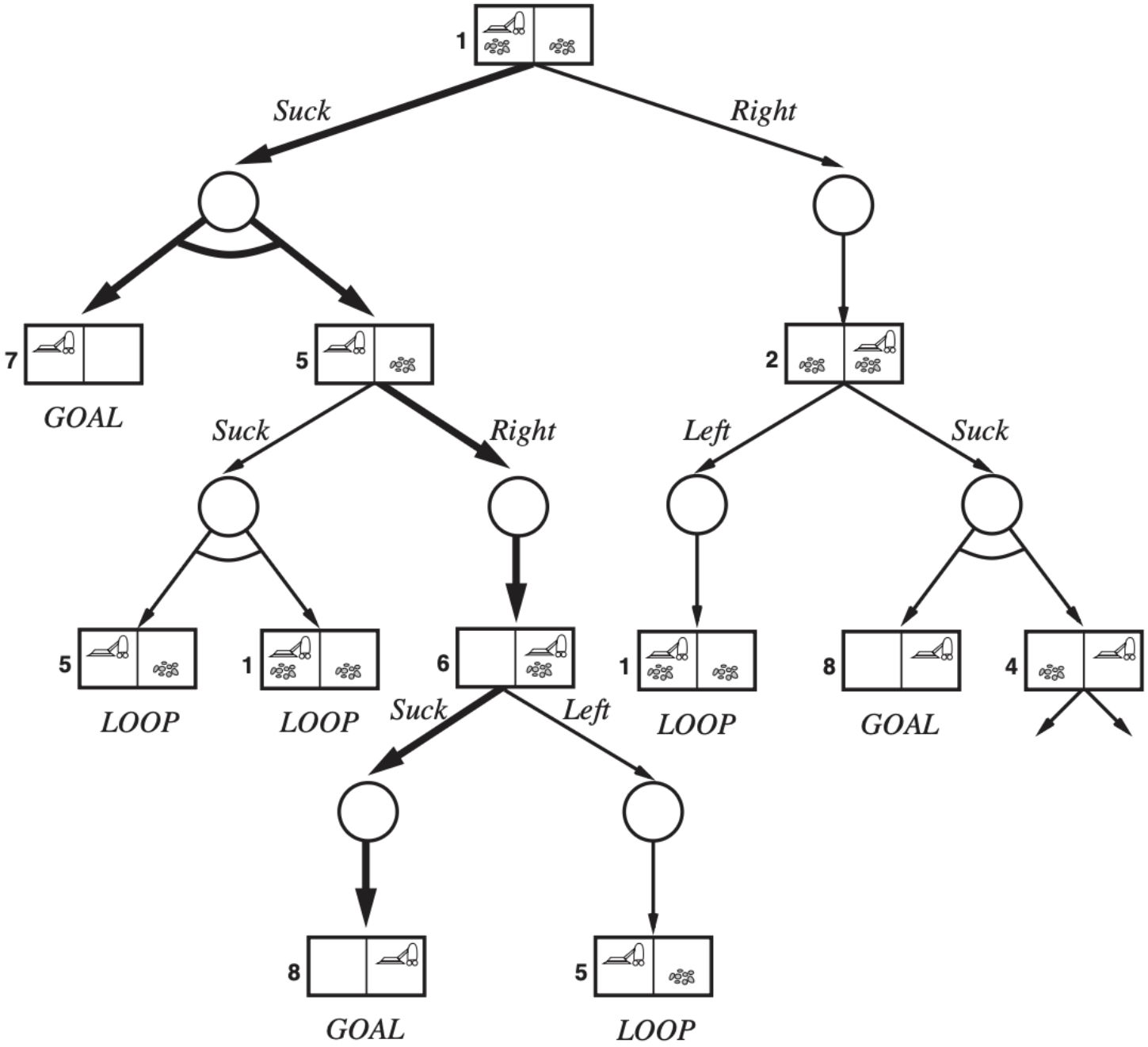
Example: From the state 1, the action Suck can lead to either the state 5 or 7.

We need a contingency plan:

[Suck, **if** State = 5 **then** [Right, Suck] **else** []]



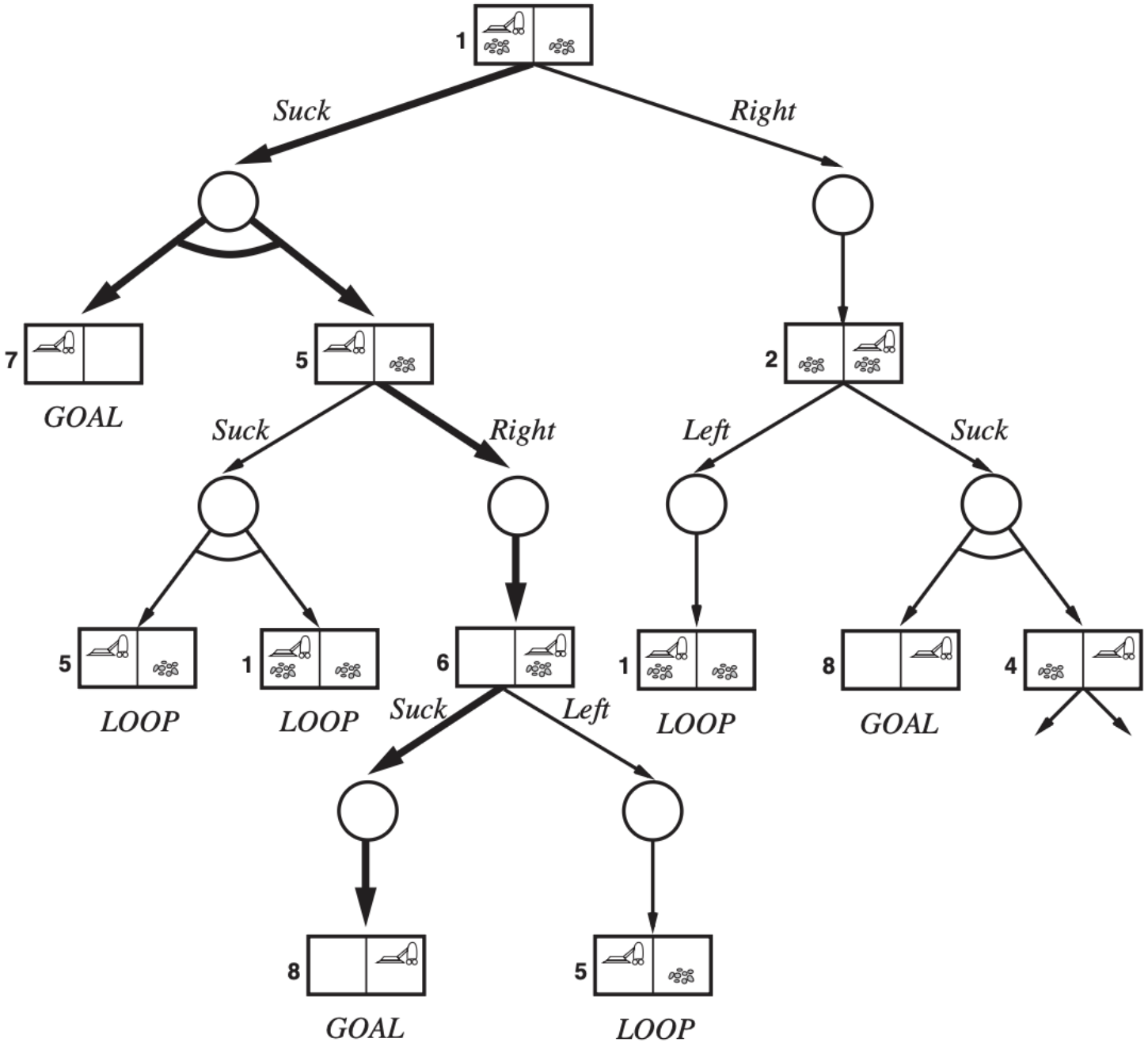
AND – OR search trees



AND – OR search trees:

- **OR nodes** represent the *agent's* own choices.
- **AND nodes** are introduced by the *environment's* choice of outcome.

AND – OR search trees



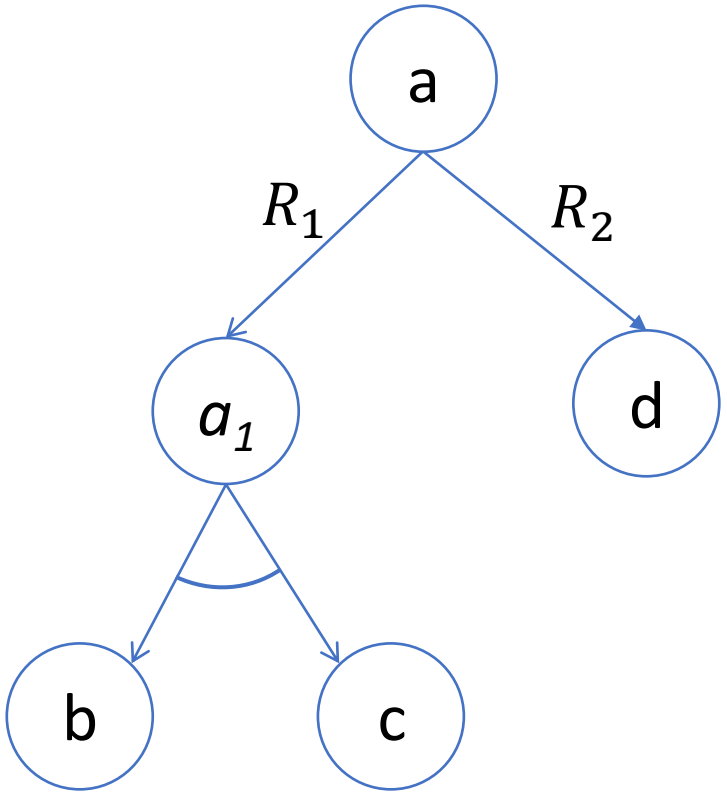
A solution is a **subtree** that:

- (1) has a goal node at every leaf.
- (2) specifies one action at each of its OR nodes.
- (3) includes every outcome branch at each of its AND nodes.

From trees to graphs

How to build an AND-OR graph:

- A node represents for each state
- If an action R transforms one state into another, e.g. R: a → b, then there is an edge from a to b.
- If R transforms one state into multiple substates, e.g. R: a → (b, c), then we introduce a new node a_1. This node represents the set of substates b, c associated with the action R.



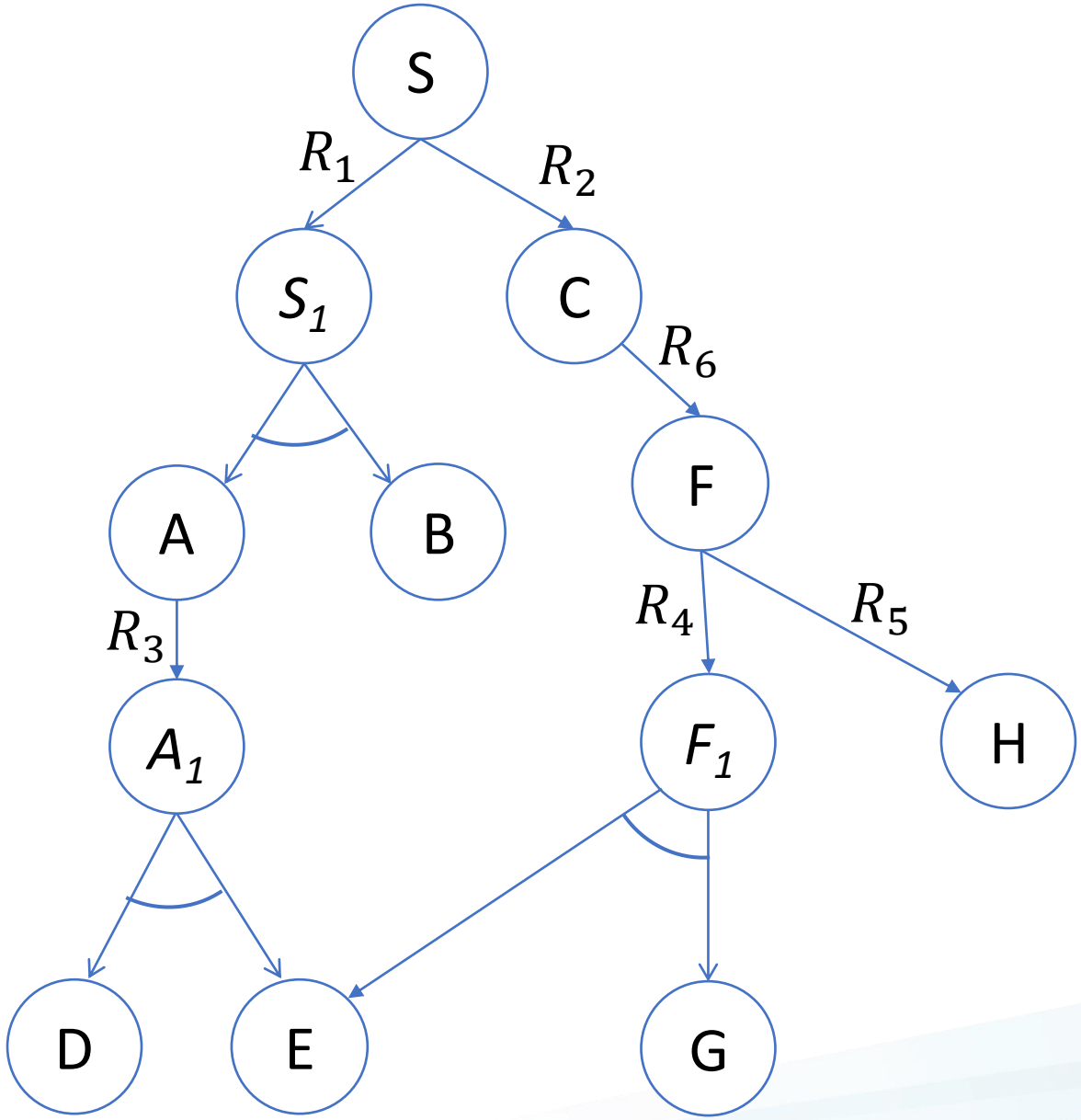
From trees to graphs

Example: Build the AND-OR graph of the following problem:

Initial state: S

Set of actions:

- R1: S → A, B
- R2: S → C
- R3: A → D, E
- R4: F → G, E
- R5: F → H
- R6: C → F



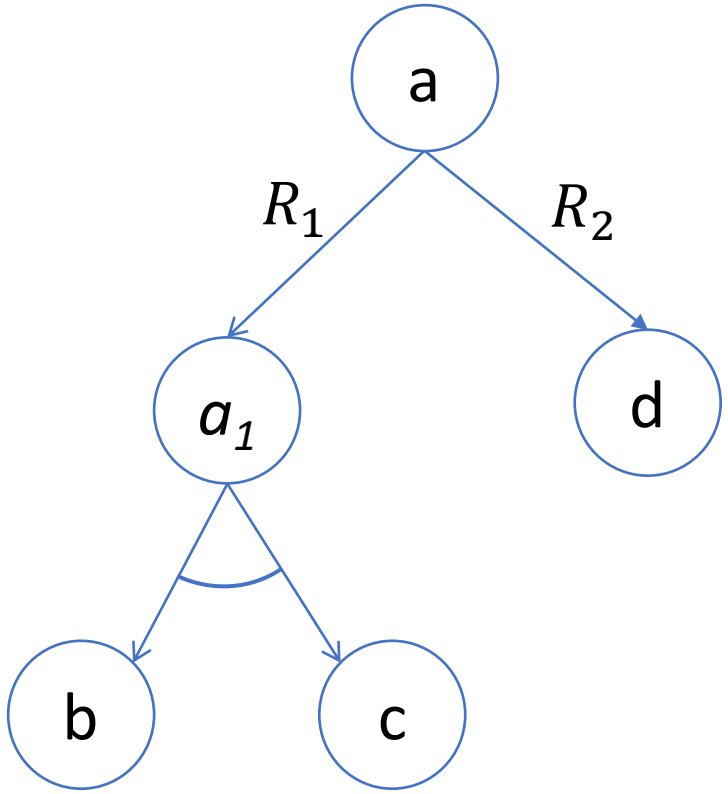
AND-OR graph search

We usually use the DFS algorithm to solve the problem.

```
function AND-OR-GRAPH-SEARCH(problem) returns a conditional plan, or failure
    OR-SEARCH(problem.INITIAL-STATE, problem, [])

---

function OR-SEARCH(state, problem, path) returns a conditional plan, or failure
    if problem.GOAL-TEST(state) then return the empty plan
    if state is on path then return failure
    for each action in problem.ACTIONS(state) do
        plan ← AND-SEARCH(RESULTS(state, action), problem, [state | path])
        if plan ≠ failure then return [action | plan]
    return failure

---

function AND-SEARCH(states, problem, path) returns a conditional plan, or failure
    for each s_i in states do
        plan_i ← OR-SEARCH(s_i, problem, path)
        if plan_i = failure then return failure
    return [if s_1 then plan_1 else if s_2 then plan_2 else ... if s_{n-1} then plan_{n-1} else plan_n]
```

AND-OR graph search

For example, consider the problem bellow where b and c are goals.



AND-OR-GRAPH-SEARCH(problem)
→ OR-SEARCH(a, problem, [])

OR-SEARCH(a, problem, [])
└— try R1
  └— AND-SEARCH({a1}, problem, [a])
    └— OR-SEARCH(a1, problem, [a])
      └— AND-SEARCH({b, c}, problem, [a1, a])
        ├— OR-SEARCH(b, problem, [a1, a]) = empty plan
        └— OR-SEARCH(c, problem, [a1, a]) = empty plan
      = success
    = success
  = success
= success

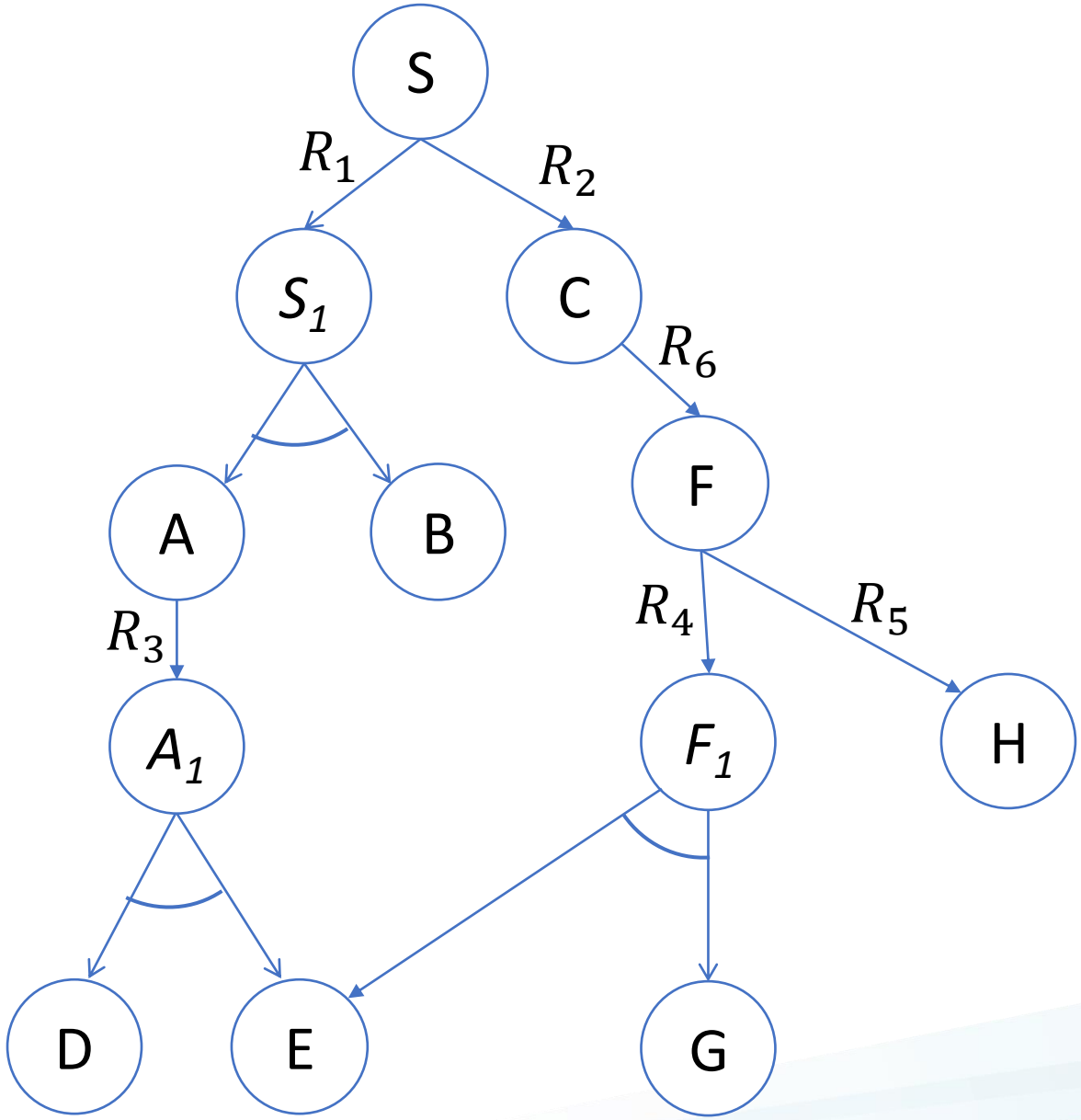
AND-OR graph search

If goal = {B,D,E,G}, is the problem solvable?

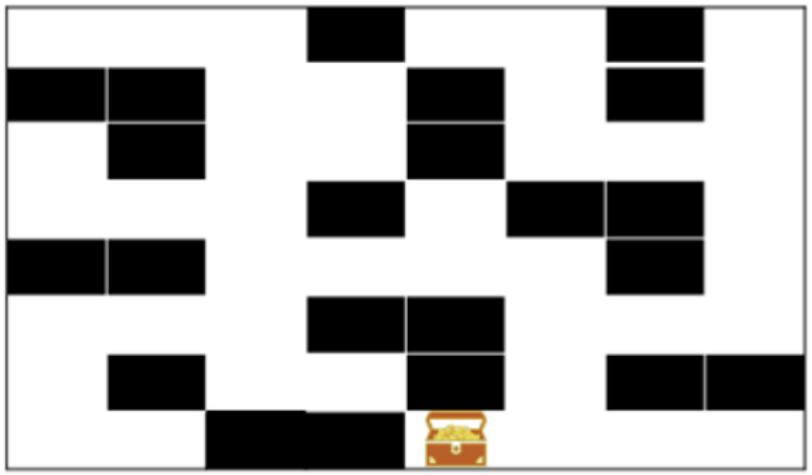
What if goal = {D,E}?

What if goal = {B,D,E}?

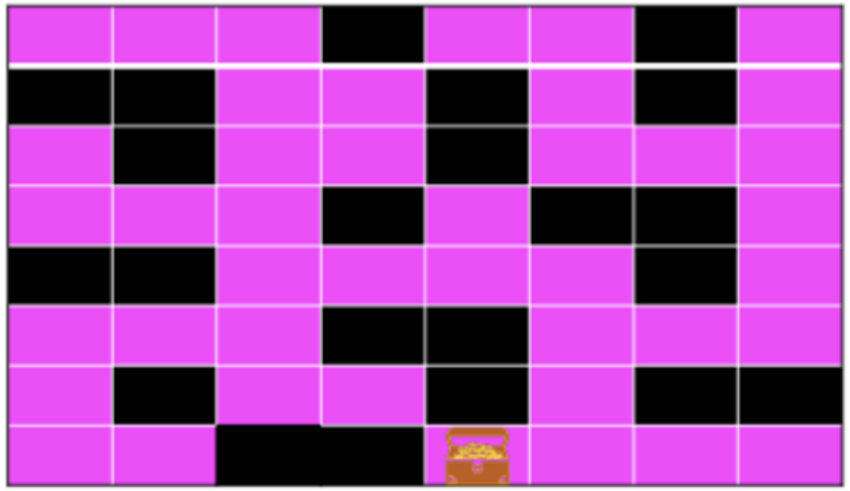
Solution = [
R1,
if state = A then R3,
if state = D then done
else if state = E then done
else if state = B then done
]



2. Searching in unobservable environments: Environments



If the environment is observable, we can solve this problem by using A* algorithm.

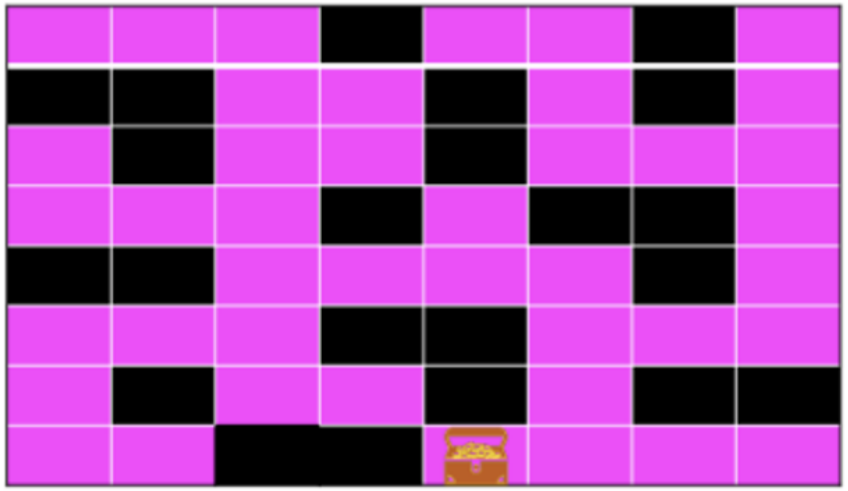


In an unobservable environment:

- We know the layout of the map (known environment)
- We don’t know the initial state
- We don’t know if we move successfully or not

Belief state

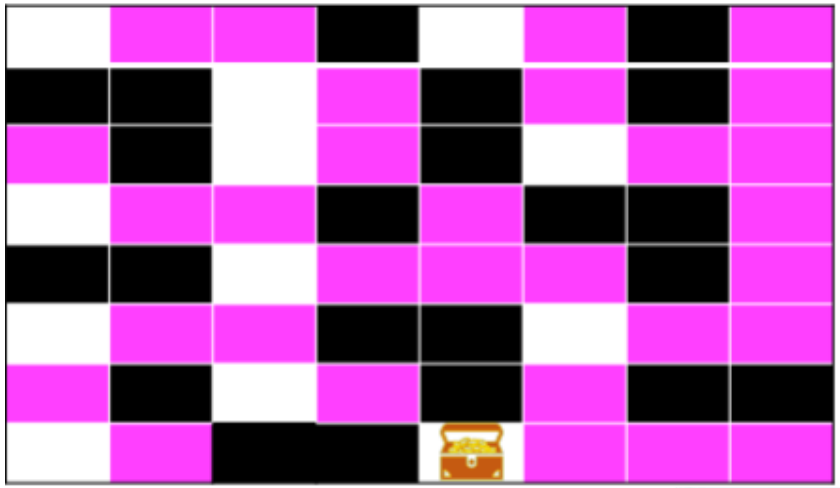
***Belief state*:** represents the agent's current belief about the possible states it might be in, given the sequence of actions and percepts up to that point.



For example:

Initially: The agent doesn't know where it is, then

belief states = all possible squares



After moving right: The agent still doesn't know its location, but it cannot be the white squares.

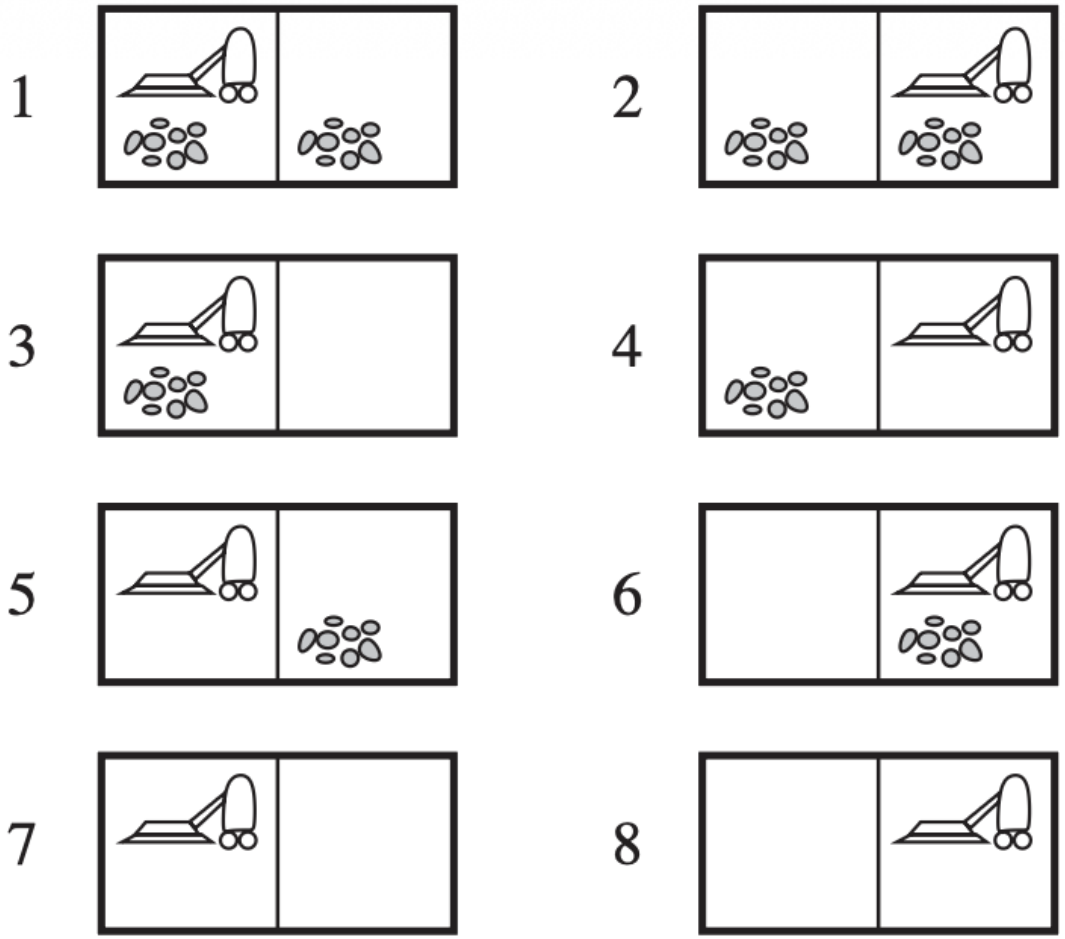
belief states = all possible pink squares

Solutions

Searching without observation = sensorless = conformant.

Example in vacuum world with no sensors:

- Agent knows the layout of the environment.
- Agent doesn't know its location and distribution of dirt
- The solution could be
  - [] → belief states = {1,2,…,8}
  - [Right] → belief states = {2,4,6,8}
  - [Right, Suck] → belief states = {4,8}
  - [Right, Suck, Left] → belief states = {3,7}
  - [Right, Suck, Left, Suck] → belief states = {7}



From physical problem to belief-state problem

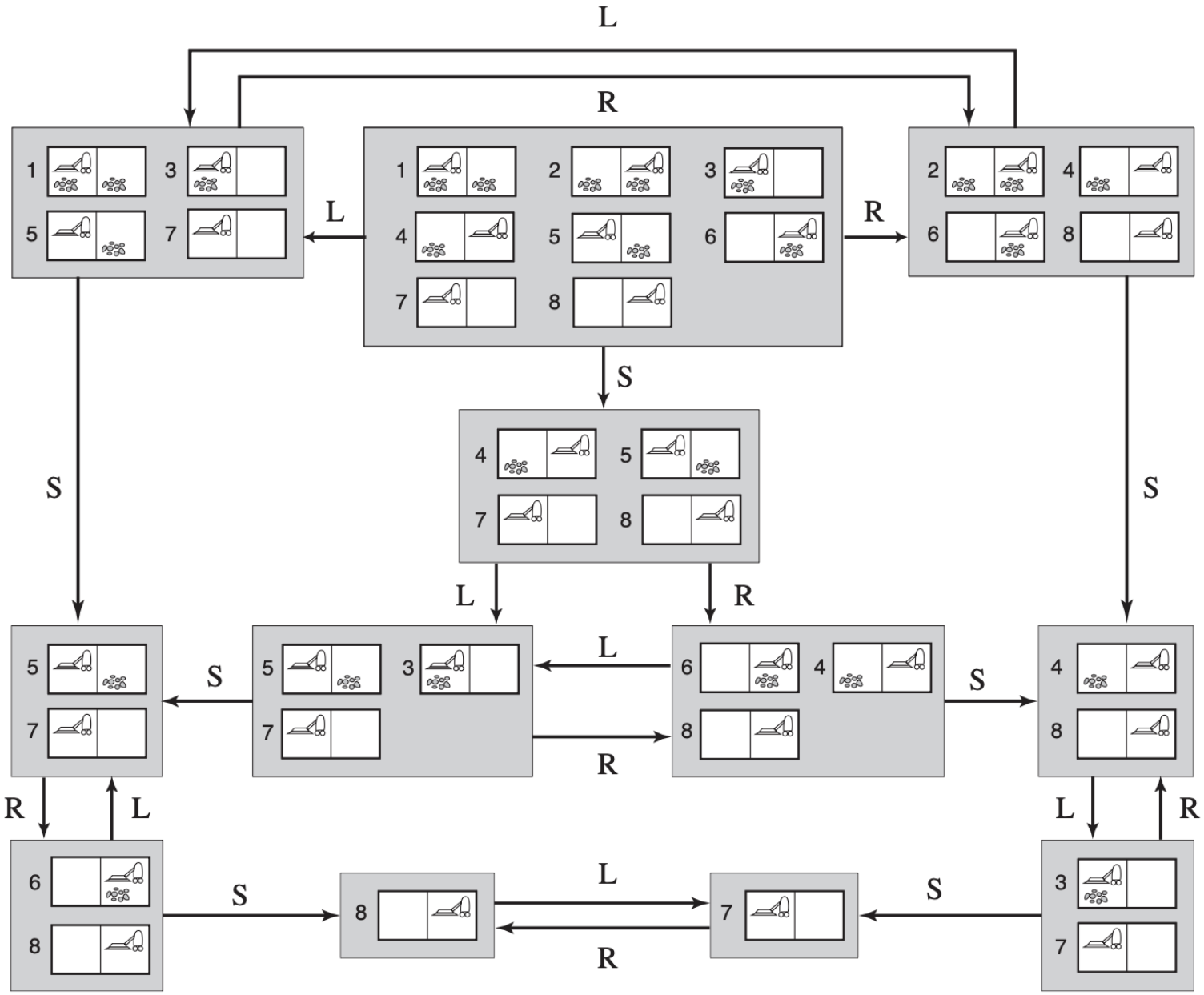
Assume the physical problem P (with N states) is defined by *ACTIONS, RESULT, GOAL-TEST.*

- **Belief states:** every possible set of N state => 2^N states.
- **Initial state:** the set of all states in P.
- **Actions:** typically union of all the actions

  ACTIONS(belief state) = UNION(*ACTIONS*(state))
- **Transition (prediction):**

  b’ = RESULT(b, a) = {s’ : s’ = *RESULT*(s, a) and s ∈ b}
- **Goal test:** a belief state satisfies the goal only if all the physical states in it satisfy *GOAL-TEST.*

Searching in unobservable environments

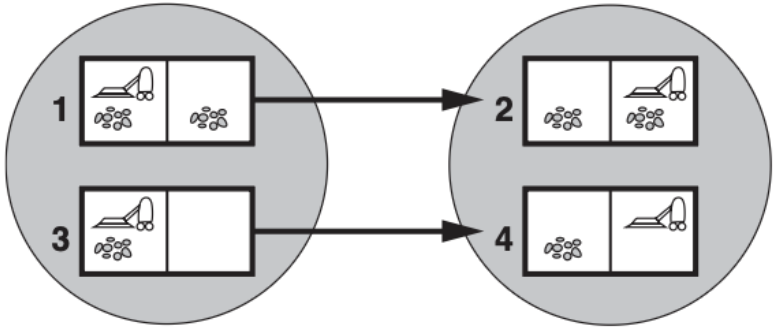


Original (physical) states

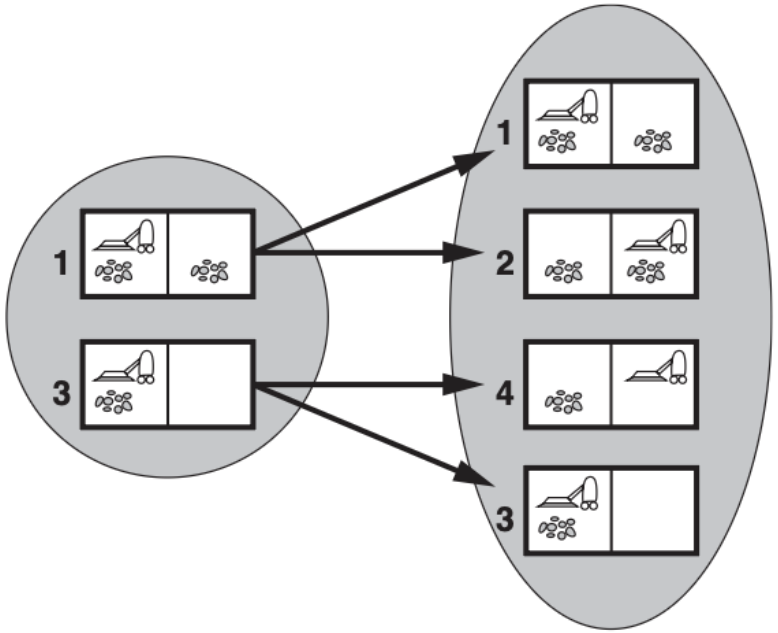
→ reachable belief states

- Reachable belief states = 12, while belief states = 256.
- We can use any search algorithm to solve this problem.

Unobservable and stochastic environments



Predicting the next belief state for the sensorless vacuum world with a deterministic action, *Right*.



Prediction for the same belief state and action in the slippery version of the sensorless vacuum world.

3. Searching in partially observable environments: Environments

Property 1

- **Fully observable**: The agent can see all the relevant aspects of the environment needed to choose the best action.
- **Partially observable**: The agent can see only part of the environment, so it must deal with missing or uncertain information.

Partially observable environments

The formal problem specification includes a PERCEPT(s) function that returns the percept received in a given state.

- Fully observable environments:

  PERCEPT(state) = env

- Unobservable environments:

  PERCEPT(s) = null

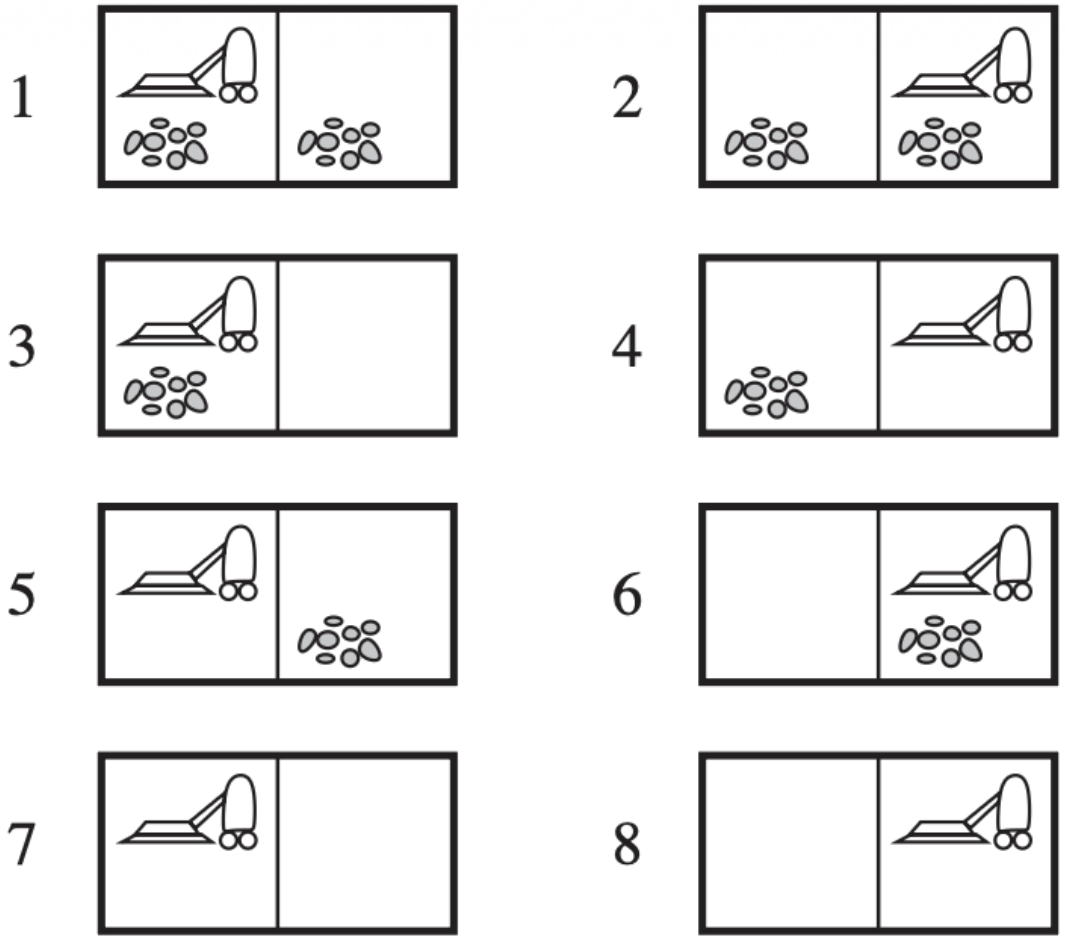
- Partially observable environments:

  PERCEPT(s) = *current percept*

  For example:

  PERCEPT(state 1) = PERCEPT(state 3) = [A, Dirty]

  PERCEPT(state 4) = PERCEPT(state 8) = [B, Clean]



Transition in partially observable environments

The same idea as unobservable environments. However, we now have percept, so the transition step can work more ”correctly.”

- **Prediction:**

$$b' = \mathit{PREDICT}(b,a)$$

- **Observation prediction:**

$$\mathit{POSSIBLE\text{-}PERCEPTS}(b') = \{o: o = \mathit{PERCEPT}(s) \text{ and } s \text{ in } b'\}$$

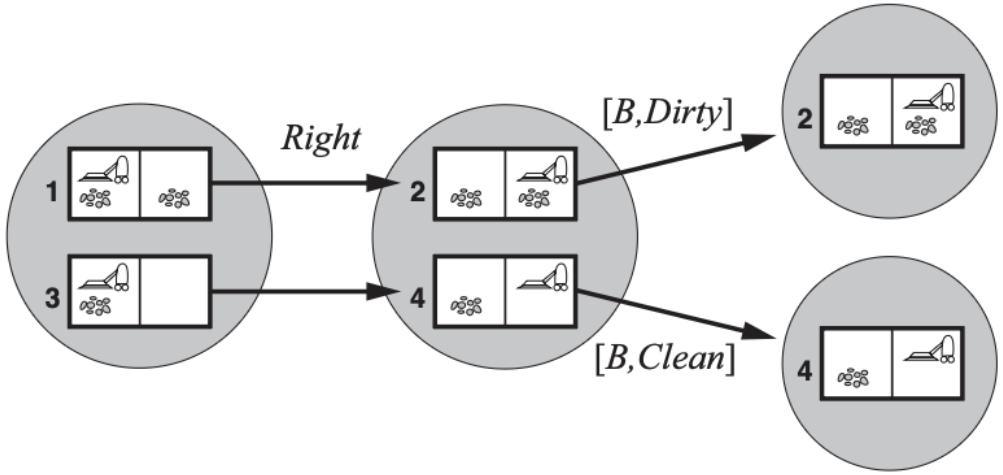
- **Update:**

$$b'' = \mathit{UPDATE}(b',o) = \{s: o = \mathit{PERCEPT}(s) \text{ and } s \text{ in } b'\}$$

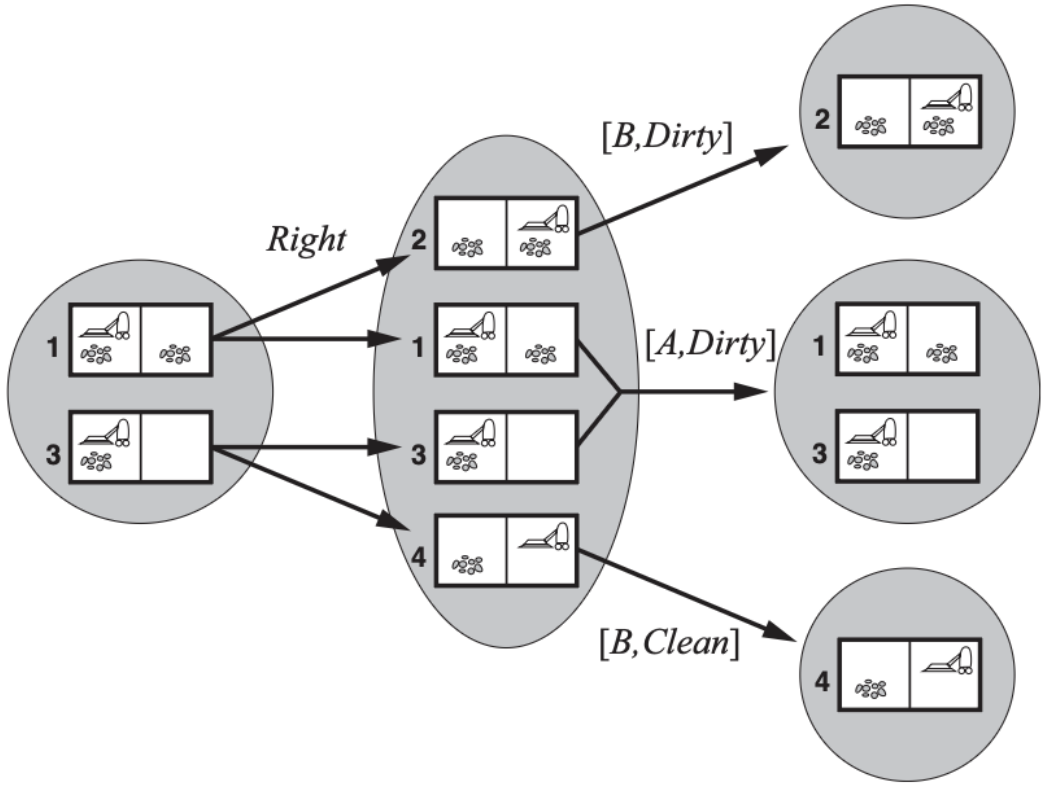
- **Transition model:**

$$\mathit{RESULT}(b,a) = \{b'': b'' = \mathit{UPDATE}(\mathit{PREDICT}(b,a),o) \text{ and } o \text{ in } \mathit{POSIBLE\text{-}PERCEPTS}(\mathit{PREDICT}(b,a))\}$$

Transition in partially observable environments

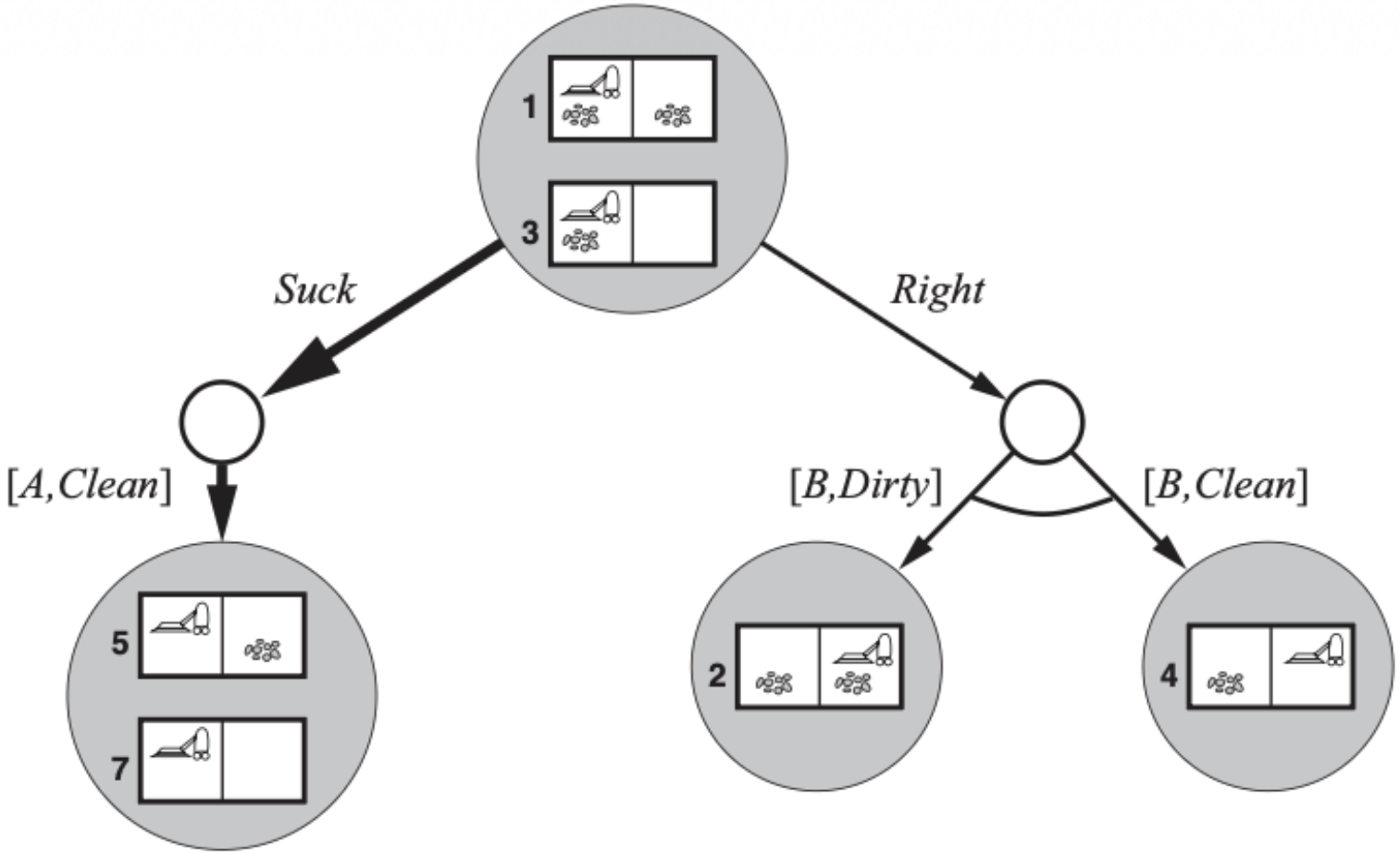


Deterministic



Non-deterministic

Searching in partially observable environments



A solution should be a condition plan rather than a sequence

Updating belief states

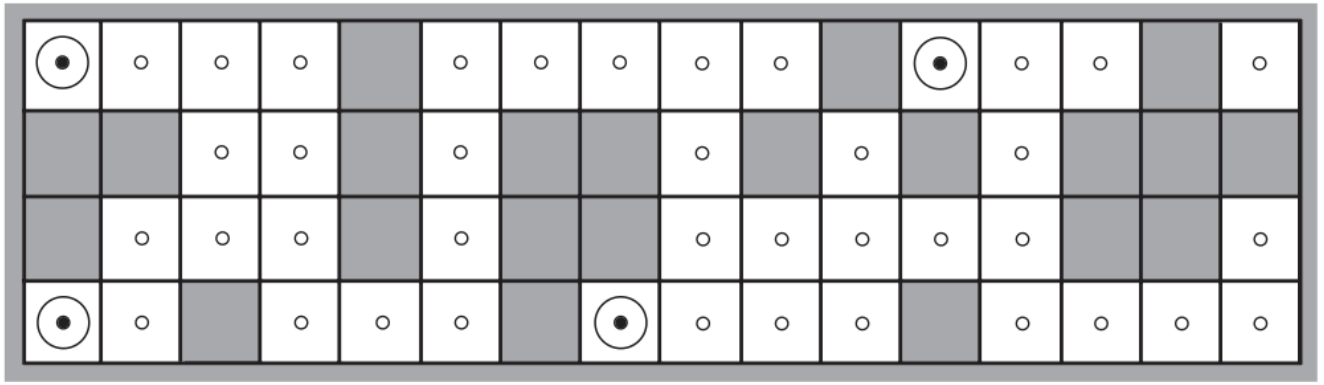
Updating this belief state over time using actions and percepts is essential for making correct decisions.

Updating strategy: b' = *UPDATE*(*PREDICT*(b, a),o)

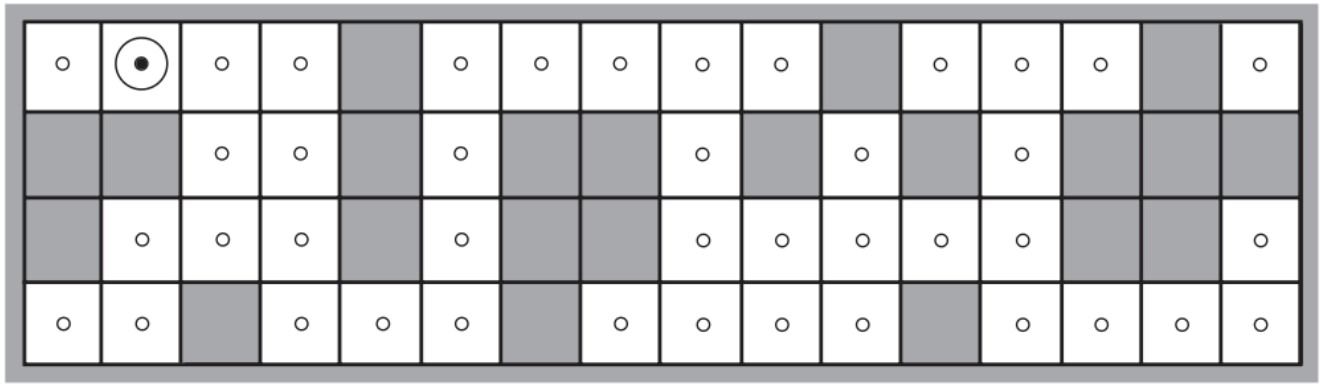
Example of a robot localization.

- A robot doesn't know where it is.
- It has 4 sensors to know whether there is an obstacle.
- Its movements are nondeterministic.

Where is the robot?



(a) Possible locations of robot after E_1 = NSW



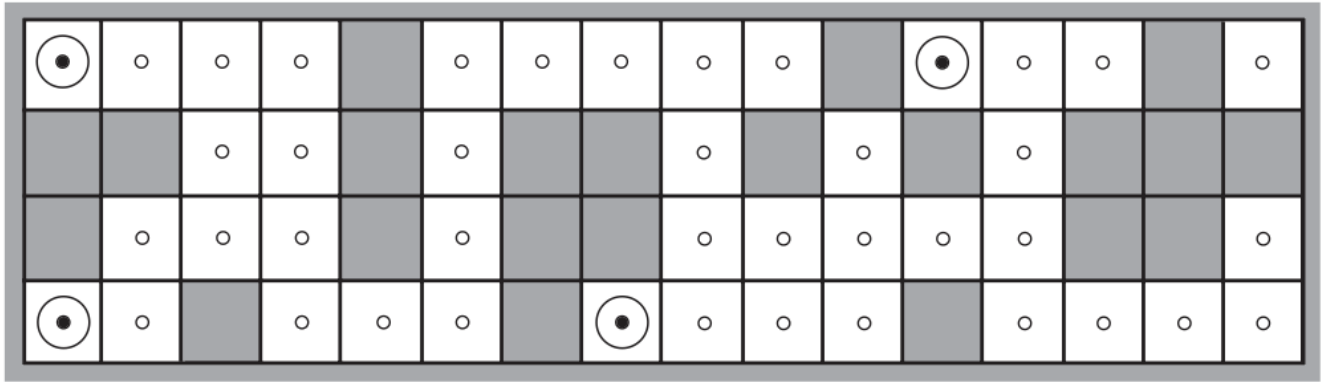
(b) Possible locations of robot After E_1 = NSW, E_2 = NS

Updating belief states

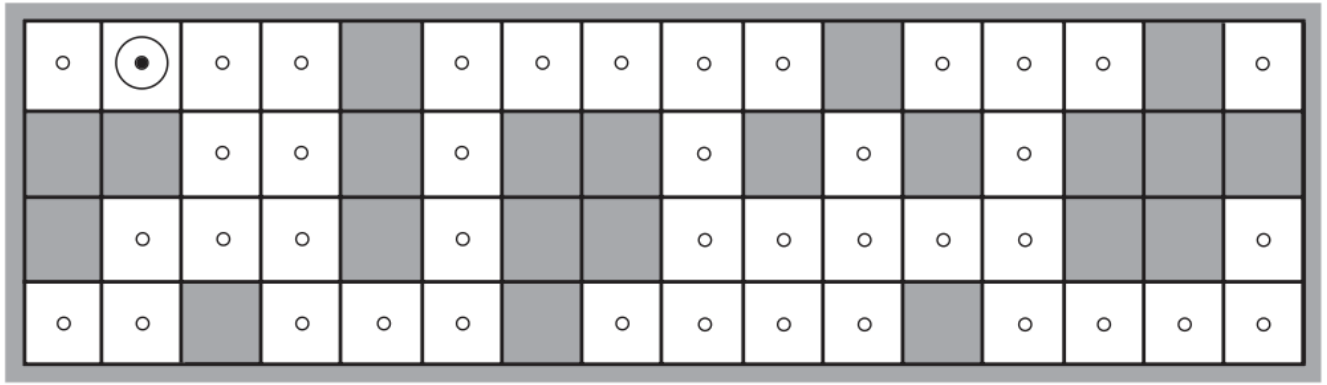
Initially, the robot perceives E1 = ESW, so there are 4 possible locations (belief state).

After moving, the belief state is non-decreasing.

After perceiving E2 = NS, the robot knows its exact location.



(a) Possible locations of robot after E_1 = NSW



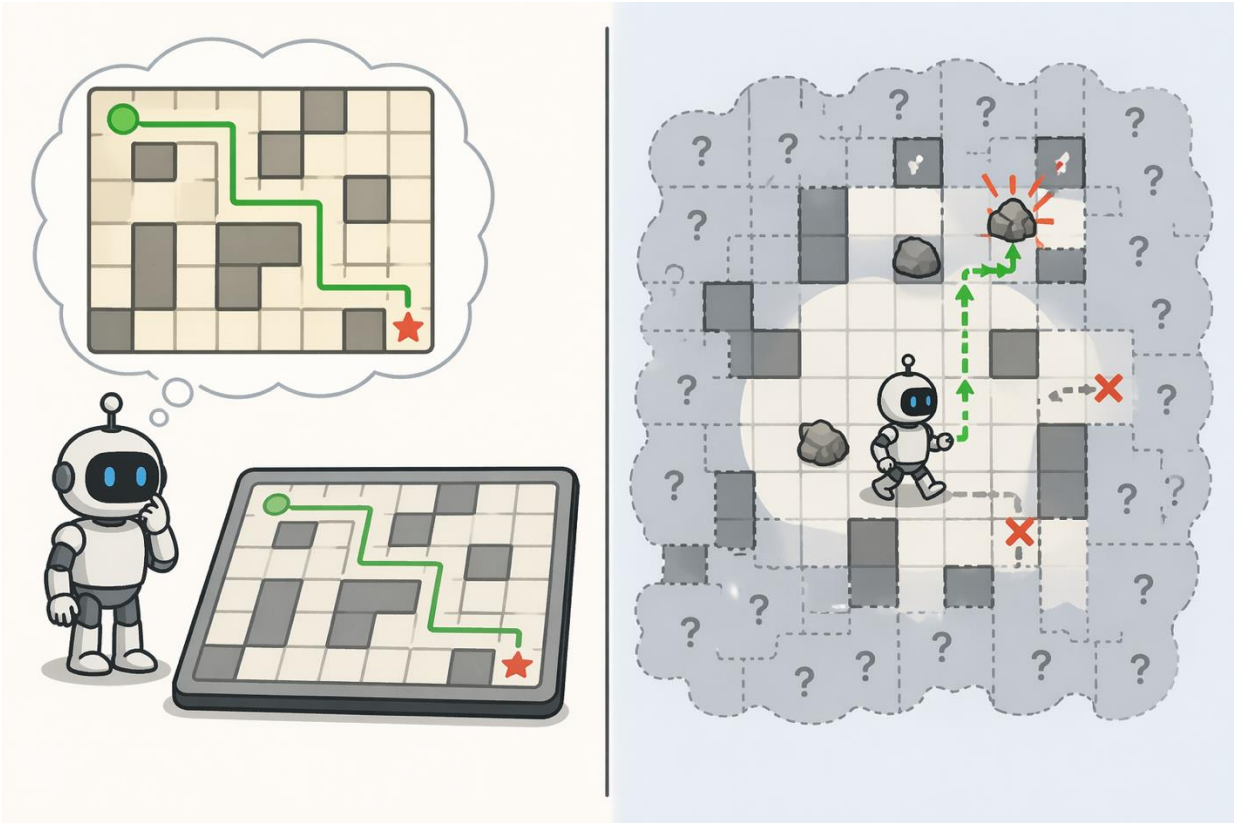
(b) Possible locations of robot After E_1 = NSW, E_2 = NS

4. Online search problems

Offline search: A complete solution is generated before anything changes in the physical world.

Online search:

- Combines computation and action
- Must solved by an agent executing actions
- Useful for dynamic environments
- Useful for nondeterministic environments, since it allows the agent to focus on contingencies that actually arise – not just those that might arise
- Useful for unknown environments

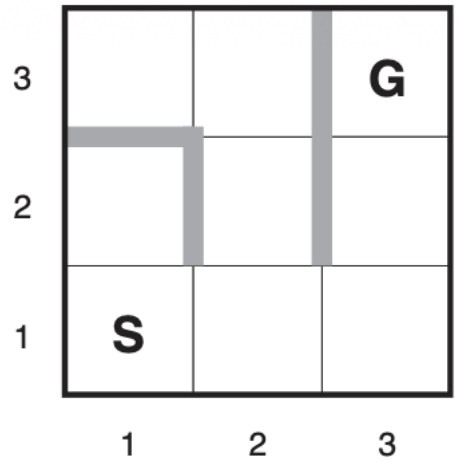


An agent only *know*:

- **Actions(*s*)**: list of actions allowed in a state *s.*
- **Step-cost function c(*s,a,s'*):** can only be determined after *s'* is explored.
- **Goal-test(*s*):** a state is goal or not.

The agent *cannot* determine RESULT(*s,a*) except by actually being in *s* and doing *a.*

The agent *might know* admissible heuristic function h(s).



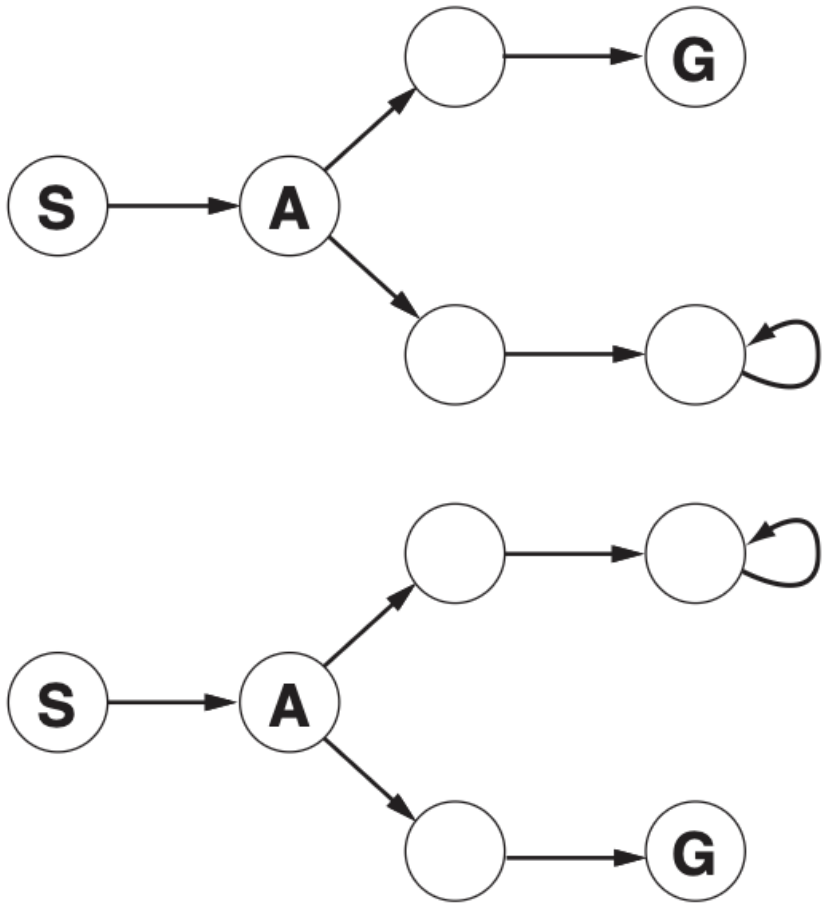
Cost: total path cost the agent actually travels.

Competitive ratio: ratio of actual cost to optimal cost.

The competitive ratio can be infinity because of:

- Irreversible actions: once taken, cannot go back, e.g., fall off a cliff!
- Dead-end state: no path to any goal, e.g., locked in a freezer.

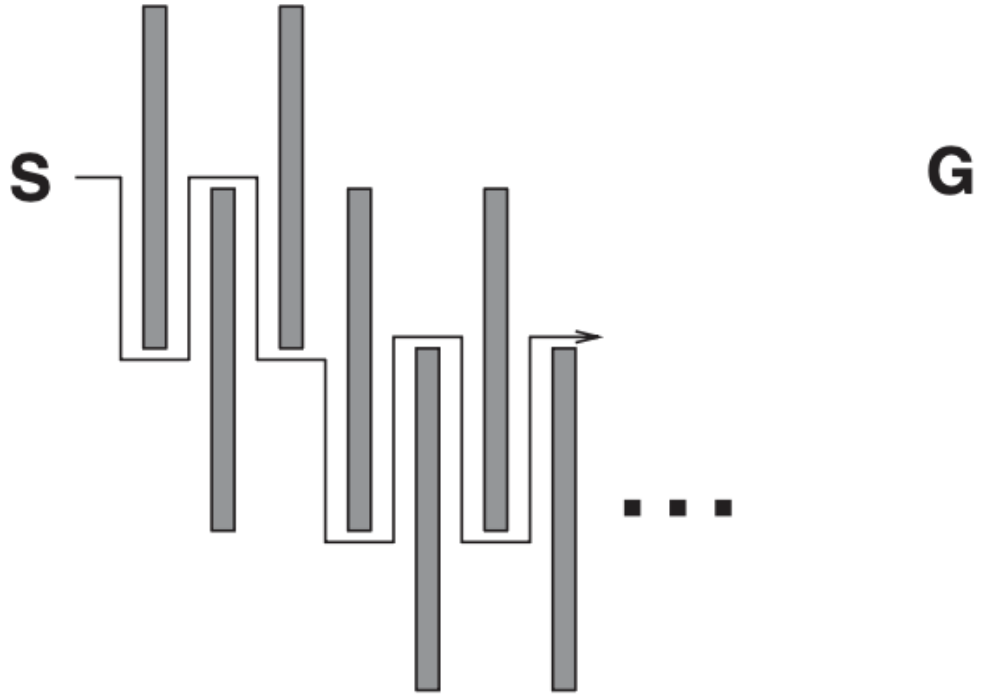
⇒ Limitations: no online algorithm can avoid this in all environments!!!



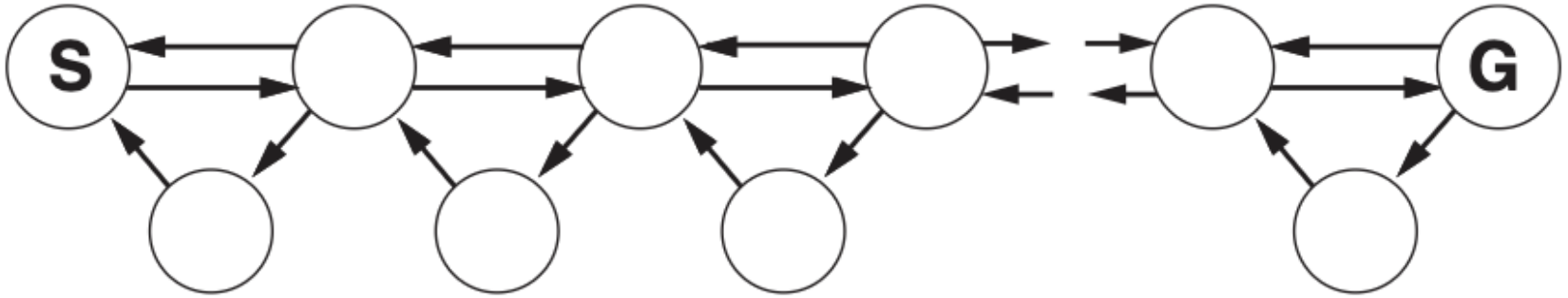
Assumption:

we simply assume that the state space is **safely explorable –** that is, some goal state is reachable from every reachable state.

For example: 8-puzzles.



However, even in safely explorable environments, **no bounded competitive ratio** can be guaranteed if there are paths of unbounded cost.



The agent can only explore from the current state

- Can't bounce around to other search paths, like offline A*
- DFS can be used, as long as actions are *reversible*
- Also, online versions of iterative deepening will work
- Random walk is possible
  - Select at random an available action from current state
  - Preference to previously unexplored actions
  - Will eventually find goal, if space is finite
  - But slow – exponential search in worst case

Exercises

Given a search problem with non-deterministic environments as follow:

Initial state: S

Set of actions:

- R1: S → A, B, C
- R2: A → D
- R3: B → E, F
- R4: C → D, G
- R5: F → H

Goal states: D, G, H

a) Build an AND-OR graph

b) Is this problem solvable? If any, find a condition plan.

Trung tâm Học liệu và Dạy học số

Đại học Công nghệ Kỹ thuật Tp. Hồ Chí Minh

01 Võ Văn Ngân, P. Thủ Đức, Tp.Hồ Chí Minh

daotaotuxa@hcmute.edu.vn